ใบงานครั้งที่ 1 ส่วน B · วางแผนก่อนทดสอบ

ENGSE601 · TeamName: TheOneBakery · Reconstructed in the requested Section B scope

ขั้นที่ 1 · เลือก feature และกฎสำคัญ

หัวข้อ	คำตอบ
ทีม / โครงงาน	TheOneBakery / THE ONE Bakery
Feature	การบันทึกคำสั่งซื้อ
กฎสำคัญ	เมื่อมีการสั่งซื้อ ระบบต้องบันทึกข้อมูลคำสั่งซื้อและคำนวณยอดรวมให้ถูกต้อง.
พบกฎที่ไหน	ใบงาน S1-B ของทีมระบุเอกสาร SRS แต่ยังไม่มีไฟล์ SRS พร้อม version อยู่ใน repo ที่ตรวจ; จึงใช้อ้างเป็น source statement จากใบงาน ไม่อ้างเป็น approved SRS.
เหตุผลที่เลือก	ยอดรวมและการบันทึกออเดอร์เป็นข้อมูลหลักของกระบวนการร้าน ความผิดพลาดกระทบทั้งลูกค้า การรับออเดอร์ และขั้นตอนผลิต/แพ็ก.

ขั้นที่ 2 · Component / Data Flow (สร้างย้อนหลัง)

ลำดับ	ชั้น / component	ข้อมูลและหน้าที่
1	Customer UI: public/index.html และ public/cart.html	ลูกค้าเลือกสินค้า/จำนวน ตรวจรายการและกรอกข้อมูลผู้สั่งซื้อ.
2	HTTP API: POST /api/orders	รับ customer_name, phone และ items จาก client.
3	Backend: routes/orders.js	ตรวจข้อมูลพื้นฐาน; อ่านราคาสินค้า active จาก DB; คำนวณ total ฝั่ง server; ตรวจ capacity/วัตถุดิบ; บันทึก order/items/status history/notification ใน transaction.
4	Data layer: PostgreSQL schema	ตาราง products, orders, order_items, recipes, inventory, order_status_history, notifications และ inventory_movements.
5	Response / follow-up	API ตอบ order id/number/status; ลูกค้าสามารถไปหน้าติดตามออเดอร์. ต้องทดสอบ runtime เพื่อยืนยัน end-to-end.

ที่มาของ flow: สร้างย้อนหลังจาก source code ใน https://github.com/Nanthawat67/the-one-bakery-New · ตรวจเมื่อ 23 ต.ค. 2569 · ไม่ใช่ diagram ที่ยืนยันว่ามีก่อนพัฒนา.

ขั้นที่ 3 · ตอบคำถามสามข้อ

คำถาม	คำตอบและเหตุผล
1. ใครรับผิดชอบ?	Backend route ใน routes/orders.js เป็นจุดที่อ่านราคา products.price และคำนวณ total ฝั่ง server ก่อนบันทึก orders/order_items. UI ใช้แสดงผลได้ แต่ไม่ควรเป็นแหล่งราคาที่เชื่อถือได้เพียงจุดเดียว.
2. ข้อมูลส่งถึงครบไหม?	เส้นทางที่ตรวจรับ customer_name, phone และ items; อ่าน product_id/quantity แล้วบันทึก order และ order_items. จาก static code ยังยืนยัน runtime persistence/ความครบทุก field ไม่ได้ ต้องตรวจ response และข้อมูลใน test DB หลังเรียก API.
3. ถ้าพัง จะรู้ได้ไหม?	catch path ของ route ตอบ JSON error และบันทึก error ทาง server; แต่ error path บางกรณีอาจคืน 500 แม้เป็น input validation. ต้องทดสอบแต่ละกรณีและบันทึก status/message จริง.

ขั้นที่ 4 · ข้อค้นพบ (4 ส่วน)

ส่วน	ข้อค้นพบ
1. โค้ดเขียนว่าอะไร / อยู่ตรงไหน	routes/orders.js ใน POST /api/orders ตรวจ quantity < 1 แต่จาก route ที่ตรวจไม่เห็น explicit integer validation ก่อน INSERT; schema กำหนด order_items.quantity เป็น INT.
2. ขัดกับข้อใด	ไม่สามารถยืนยันว่าเป็นการละเมิด SRS เพราะยังไม่มี SRS/version ที่อนุมัติให้ตรวจเทียบ; เป็นช่องว่างของ input validation ที่เกี่ยวข้องกับกฎหลักเรื่องการบันทึกออเดอร์และคำนวณยอดรวม.
3. ผลต่อการทดสอบ	ส่ง quantity แบบเศษส่วน เช่น 1.5 อาจไปถึงชั้นฐานข้อมูลและเกิด error; คาดการณ์นี้เป็น inference จาก static review ยังไม่ได้พิสูจน์ด้วย runtime.
4. ข้อเสนอ	ยืนยันกฎ quantity กับ owner; เพิ่ม validation จำนวนเต็มบวกที่ API ก่อนเริ่ม transaction/INSERT; ทดสอบค่าขอบ 1, 0, -1, 1.5 และตรวจว่าไม่เกิด partial write.

สถานะ finding: static source observation + inference. ไม่ใช่ผลการทดสอบ runtime หรือข้อสรุปว่าเกิด failure แล้ว.

ขั้นที่ 5 · ขอบเขตและกลยุทธ์การทดสอบ (3.5.1)

ทดสอบ	ขอบเขต / วิธี
การส่งคำสั่งซื้อ	ทดสอบ POST /api/orders ด้วยข้อมูล valid/invalid; ตรวจ status และ order number.
การคำนวณยอดรวม	เทียบ total ฝั่ง server กับผลรวมราคา authoritative × quantity ของทุก item.
การบันทึกคำสั่งซื้อ	ตรวจ row ใน orders และ order_items รวมถึง status history หลังบันทึก.
จำนวนสินค้า / ราคา / หลายรายการ	ทดสอบ quantity ที่ถูกและขอบ, product ไม่พบ/ไม่ active, หลายสินค้า และตรวจว่าราคาใช้จากฐานข้อมูล.
ตรวจหลังบันทึก	ตรวจ response กับข้อมูลที่เก็บจริงในฐานข้อมูล test; ทดสอบกรณีล้มเหลวว่ามี partial write หรือไม่.
ไม่ทดสอบใน scope นี้	ไม่เน้นรายละเอียด visual styling/cross-browser ที่ไม่ใช่เป้าหมายหลักของ feature; ไม่ได้หมายความว่า UI ทั้งหมดไม่มี defect.

ระดับและความเสี่ยง

รายการ	คำตอบ
ระดับที่เลือก	Unit + Integration/API + System ตามที่ระบุในคำตอบส่วน B ของทีม.
ความเสี่ยง 1	ยอดรวมคำนวณผิด — กระทบยอดที่ลูกค้าต้องจ่ายและรายรับ.
ความเสี่ยง 2	บันทึกคำสั่งซื้อไม่ครบ — กระทบการรับออเดอร์และการผลิต.
ความเสี่ยง 3	ข้อมูลที่แสดงหลังสั่งซื้อไม่ตรงกับข้อมูลในฐานข้อมูล — กระทบความเชื่อถือและการติดตามสถานะ.

ขั้นที่ 6 · Entry และ Exit criteria (วัดได้)

ประเภท	เกณฑ์
Entry 1	Application และ PostgreSQL test database เริ่มทำงานได้ และมี connection ที่ใช้ทดสอบได้.
Entry 2	มี product test data พร้อมราคาและสถานะ active ที่ทราบค่า expected; ใช้ข้อมูลสังเคราะห์ ไม่ใช่ข้อมูลลูกค้าจริง.
Exit 1	กรณีทดสอบที่กำหนดไว้ทั้งหมดมีสถานะจริง Pass/Fail/Blocked/Not Executed และลิงก์หลักฐาน; ห้ามนับ Not Executed ว่า Pass.
Exit 2	ไม่มี defect ระดับสูงเกี่ยวกับการบันทึกคำสั่งซื้อหรือคำนวณยอดรวมที่ยังไม่ตัดสินใจ; หากมีต้องระบุ owner/การตัดสินใจก่อน release.

ขั้นที่ 7 · หน้าที่สมาชิกและการใช้ AI

ชื่อสมาชิกจากใบงานเดิม	หน้าที่จากใบงานเดิม	GitHub username
นันทวัฒน์ แซ่ย่าง	Test Scope, Strategy, Risk, Entry/Exit	@Nanthawat67
ธนกร ผดุงศิลป์	Feature และ Business Rule	[ให้ผู้ใช้ยืนยัน/เติม]
อภิรยา สายนาคา	Component และ Data Flow	[ให้ผู้ใช้ยืนยัน/เติม]
ณัฐพงศ์ จันทร์สิงห์	ตรวจสอบกฎและข้อค้นพบ	[ให้ผู้ใช้ยืนยัน/เติม]

AI declaration: ใช้ AI ช่วยจัดโครงสร้างและทบทวนความเสี่ยงของ quantity validation จาก worksheet/source code. สิ่งที่รับ/แก้/ปฏิเสธต้องให้ทีมยืนยันและบันทึกใน AI-LOG.md; ไม่มีการสร้างผล runtime หรือ screenshots. Student IDs ถูกละเว้นเพราะ repo เป็น Public.